

GENERATIVE AI

FAQ BOT project

NAME: KRITHIKA A
SRN: PES2UG23CS924
SEC: E

PROBLEM STATEMENT:

Automated FAQ Bot

Goal: Customer support bot that answers questions based on a company's policy document.

Tech: QA pipeline using the policy text as context.

ABSTRACT:

Finding answers in long company policy documents are very time taking and difficult. FAQ bot allows users to find accurate answers to their queries using transformer based Question answering. The system reads the company policy file (.txt) give separately and allows the user to ask question in natural language. the bot extracts the most relevant answer directly from the document instead of generating new text using distilbert-base-cased-distilled-squad model. The project demonstrates a simple application of modern Natural Language Processing techniques for improving customer support.

SHORT DOCUMENTATION:

Through this project, I learned how transformer-based models can understand and process natural language using attention mechanisms, and how extractive Question Answering works by selecting the most relevant answer span directly from a given document instead of generating new text. I also understood the importance of clear question phrasing and well-structured documents for achieving accurate results. Based on this understanding, I built an Automated FAQ Bot that answers user queries using a company policy document stored in an external text file. The system uses Hugging Face's question-answering pipeline with the distilbert-base-cased-distilled-squad model to extract precise answers from the document and return them along with a confidence score. This project demonstrates a simple and effective application of modern NLP techniques for automating customer support and improving information accessibility.

OBSERVATION:

```
[5]: with open("SampleCompanyPolicy.txt", "r", encoding="utf-8") as file:
      context = file.read()
      print("Policy document loaded successfully!")

Policy document loaded successfully!

[2]: from transformers import pipeline
      qa_bot = pipeline(
          "question-answering",
          model="distilbert-base-cased-distilled-squad"
      )

config.json: 100% ██████████ 473/473 [00:00<00:00, 29.0kB/s]
C:\Users\ariva\AppData\Roaming\Python\Python313\site-packages\huggingface_hub\file_download.py:130: UserWarning: `huggingface_hub` cache-system uses symlinks by default to efficiently store duplicated files but your machine does not support them in C:\Users\ariva\.cache\huggingface\hub\models--distilbert-base-cased-distilled-squad. Caching files will still work but in a degraded version that might require more space on your disk. This warning can be disabled by setting the `HF_HUB_DISABLE_SYMLINKS_WARNING` environment variable. For more details, see https://huggingface.co/docs/huggingface_hub/how-to-cache#limitations.
To support symlinks on Windows, you either need to activate Developer Mode or to run Python as an administrator. In order to activate developer mode, see this article: https://docs.microsoft.com/en-us/windows/apps/get-started/enable-your-device-for-development
  warnings.warn(message)
model.safetensors: 100% ██████████ 261M/261M [00:10<00:00, 19.1MB/s]
Loading weights: 100% ██████████ 102/102 [00:00<00:00, 1022.65it/s, Materializing param=qa_outputs.weight]
tokenizer_config.json: 100% ██████████ 49.0/49.0 [00:00<00:00, 3.11kB/s]
vocab.txt: 100% ██████████ 213k/213k [00:00<00:00, 4.57MB/s]
tokenizer.json: 100% ██████████ 436k/436k [00:00<00:00, 773kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
```

Hardcoded question:

```
[6]: question = "How many days of paid leave are employees entitled to?"
      result = qa_bot(question=question, context=context)

      print("Question:", question)
      print("Answer:", result["answer"])
      print("Confidence Score:", round(result["score"], 2))

Question: How many days of paid leave are employees entitled to?
Answer: 12
Confidence Score: 0.46

[9]: q = "Who should salary discrepancies be reported to?"
      res = qa_bot(question=q, context=context)
      print("Question: ", q)
      print("Answer: ", res["answer"])
      print("Confidence Score: ", round(res["score"], 2))

Question: Who should salary discrepancies be reported to?
Answer: HR
Confidence Score: 1.64
```

Question at runtime:

```
[21]: while True:
        question = input("Ask your question: ")
        if question.lower() == "exit":
            print("Exiting FAQ Bot. Thank you!")
            break
        result = qa_bot(question=question, context=context)
        print("\nAnswer:", result["answer"],)
        print("Confidence Score:", round(result["score"], 2), "\n")
        if result["score"] < 0.3:
            print("Answer not confidently found in the document.\n")

Ask your question: What are the customer support hours?

Answer: 10:00 AM to 6:00 PM IST
Confidence Score: 0.62

Ask your question: exit
Exiting FAQ Bot. Thank you!

[ ]:
```